Année 2025-2024

Projet de fin de module en structures de données

SIMULATION D'UNE FERME INTELLIGENTE

Projet réalisé par :

Mehdad Hajar
Niaaniaa Salma
Nora El-Fettouchi

Projet encadré par :

M. Abdelwahab Naji

SOMMAIRE

01 Introduction

02 Définition des Structures de Données

03 Prototypes des Fonctions Utilisées

04 Difficultés Rencontrées

05 Perspectives

INTRODUCTION

Contexte du projet.

L'agriculture est un secteur vital pour répondre aux besoins alimentaires de la population mondiale croissante. Cependant, les défis liés à l'efficacité des ressources, comme l'eau et l'énergie, ainsi que les conditions climatiques imprévisibles, rendent la gestion des fermes de plus en plus complexe. Dans ce contexte, les technologies d'agriculture intelligente, qui utilisent des capteurs et des systèmes automatisés, offrent une solution prometteuse. Notre projet s'inscrit dans ce cadre en cherchant à développer un système de gestion de ferme basé sur des capteurs connectés et des algorithmes de gestion des ressources.

Définition du problème :

Dans les fermes traditionnelles, il est souvent difficile de surveiller et de gérer efficacement les ressources telles que l'eau et l'énergie. Par exemple, l'irrigation peut être activée même lorsque le sol est déjà suffisamment humide, ce qui entraîne un gaspillage d'eau. De plus, les agriculteurs doivent souvent se rendre physiquement sur place pour vérifier les conditions des champs, ce qui est inefficace en termes de temps et d'efforts. Ce projet vise à résoudre ce problème en développant une solution automatisée et connectée basée sur des capteurs pour surveiller l'humidité, la température et d'autres variables, afin de réduire le gaspillage de ressources et d'améliorer l'efficacité.

Motivations et intérêt pratique :

Ce projet présente un intérêt pratique significatif dans le domaine de l’agriculture moderne, notamment avec l’évolution vers des fermes intelligentes. En permettant l'automatisation de certains processus (comme l'irrigation ou l'activation du chauffage), ce projet aide à optimiser les ressources en fonction des variations climatiques. Il représente également une approche innovante pour comprendre et gérer les conditions environnementales qui affectent la production agricole, permettant une gestion plus efficace des fermes et la réduction des coûts liés aux interventions manuelles.
De plus, l'utilisation de modèles prédictifs et l'intégration de systèmes automatisés offrent des possibilités d’amélioration dans la gestion des cultures et des ressources énergétiques, contribuant ainsi à une meilleure durabilité et rentabilité des exploitations agricoles.

DÉFINITION DES STRUCTURES DE DONNÉES

Le projet utilise plusieurs structures de données pour gérer les différents composants d'une ferme intelligente. Ces structures permettent une gestion efficace des capteurs, des alertes, des notifications et des systèmes de la ferme, tout en permettant une évolution et une flexibilité pour intégrer de nouvelles fonctionnalités.

les structures de données utilisées.

1.Structure capteur

```
// structure capteur ;
 typedef struct capteur{
    int ID;
    char *type;
    int val;
}capteur;
```

Rôle : Représente un capteur avec un ID, un type (mesure effectuée) et une valeur mesurée.
Justification : Permet de regrouper les informations provenant de différents capteurs dans la ferme (température, humidité, pression, etc.).
Impact : La gestion des capteurs avec des pointeurs permet une manipulation flexible et dynamique des données.

2. Structure serre et serres

```
// structure serre :
typedef struct serre{
    int ID;
    capteur *temperature;
    capteur *humidite;
    capteur *pluit;
    int chauffage;//0:active,1:desactive
    int systeme_irrigation;
    char *etat;//ouvert ou ferme
}serre;
typedef struct serres{
    serre *serre;
    struct serres *next;
}serres;
```

Rôle : Une *serre* contient des capteurs de température, humidité et pluie, ainsi que des informations sur le chauffage, l'irrigation et l'état (ouvert/fermé). *serres* est une liste chaînée permettant de gérer plusieurs serres.
Justification : Permet de gérer chaque serre de manière autonome tout en offrant une structure flexible pour le nombre variable de serres.
Impact : La liste chaînée facilite l'ajout et la suppression de serres, mais peut ralentir l'accès aux serres par rapport à un tableau simple.

3. Structure champ_ouvert et champs

```
//structure champ_ouvert :
typedef struct champ_ouvert{
    int ID;
    capteur *temperature;
    capteur *humidite;
    capteur *ph;
    int systeme_irrigation;
}champ_ouvert;
typedef struct champs{
    champ_ouvert *champ_ouvert;
    struct champs *next;
}champs;
```

Rôle : Un champ_ouvert contient des capteurs pour la température, l'humidité, le pH, et un système d'irrigation. champs est une liste chaînée pour gérer plusieurs champs.
Justification : La gestion dynamique des champs permet d'ajouter ou de supprimer facilement des champs de culture, tout en gardant la flexibilité d'accès.
Impact : Le choix de liste chaînée permet une gestion souple mais peut rendre l'accès aux champs spécifiques plus lent.

4. Structure reserve_eau et reserves_eau

```c
// structure reserve d'eau :
typedef struct reserve_eau{
    int ID;
    capteur *niveau_eau;
    capteur *fuit;//0 ou 1
    capteur *pression;
    int pompe_remplissage;
    int vanne_decharge;
}reserve_eau;
typedef struct reserves_eau{
    reserve_eau *reserve_eau;
    struct reserves_eau *next;
}reserves_eau;
```

Rôle : reserve_eau contient des capteurs pour le niveau d'eau, les fuites, et la pression, ainsi que des informations sur la pompe de remplissage et la vanne de décharge. reserves_eau est une liste chaînée de ces structures.
Justification : Permet de surveiller plusieurs réserves d'eau dans la ferme et de gérer les alertes liées à l'état de l'eau.
Impact : L'utilisation de listes chaînées permet une gestion flexible mais nécessite un suivi plus attentif des ressources mémoire.

5. Structure panneau_sol et panneaux_sol

```c
// structure panneau solaire :
typedef struct panneau_sol{
    int ID ;
    capteur *luminosite;
    char *etat_journnee;
    int ajustement_panneaux;
}panneau_sol;
typedef struct panneaux_sol{
    panneau_sol *panneau_sol;
    struct panneaux_sol *next;
}panneaux_sol;
```

Rôle : Un panneau_sol contient un capteur de luminosité et un état de la journée pour ajuster le panneau solaire. panneaux_sol gère plusieurs panneaux solaires.
Justification : Permet de gérer efficacement l'optimisation de la production d'énergie solaire en fonction de l'exposition au soleil.
Impact : La gestion dynamique des panneaux permet de s'adapter aux variations de l'ensoleillement, mais cela augmente la complexité de gestion des ressources.

6. Structure ferme

```c
// structure de la ferme :
typedef struct ferme{
    serres *serres;
    champs *champs;
    reserves_eau *reserves_eau;
    panneaux_sol *panneaux_sol;
}ferme;
```

Rôle : La structure ferme regroupe les différentes entités de la ferme, comme les serres, les champs, les réserves d'eau et les panneaux solaires.
Justification : Permet une vue d'ensemble de la ferme en rassemblant toutes les informations nécessaires à la gestion.
Impact : Une gestion efficace mais qui repose sur une manipulation complexe de structures imbriquées.

7. Structure alerte et alertes

```c
// structure alerte :
typedef struct alerte{
    int ID_element;
    char *description;
    int priorite;
}alerte;
typedef struct alertes{
    alerte *alerte;
    struct alertes *next;
}alertes;
```

Rôle : alerte contient des informations sur les alertes (élément concerné, description et priorité). alertes est une liste chaînée pour gérer l'ensemble des alertes de la ferme.
Justification : Permet de surveiller l'état des équipements et de réagir rapidement en cas d'anomalie.
Impact :La gestion par priorité dans la liste permet de traiter les alertes les plus urgentes en premier, bien que cela ralentisse l'insertion d'alertes spécifiques.

8. Structure notification et file

```c
// structure notification :
typedef struct notification{
    char *action;
    int id_zone;
} notification;
typedef struct notifications{
    notification *notif;
    struct notifications *next;
}notifications;
typedef struct file{
    notifications *first;
    notifications *last;
    int size;
}file;
```

Rôle : notification contient l'action à effectuer et l'identifiant de la zone concernée. file est une structure de file pour gérer l'ordre des notifications.
Justification : Permet de gérer de manière ordonnée les actions à entreprendre dans la ferme.
Impact : L'utilisation d'une file garantit que les notifications sont traitées dans l'ordre, mais le stockage dynamique des notifications peut entraîner une surcharge mémoire si le nombre de notifications est trop élevé.

PROTOTYPES DES FONCTIONS UTILISÉES

Fonctions de gestions des alertes :

Fonction : creat_alerte

Signature : `alerte *creat_alerte(int ID_element,char *description,int priorite)`

Rôle :
Cette fonction permet de créer une nouvelle alerte en allouant dynamiquement de la mémoire pour l'objet alerte et en initialisant ses champs.

Entrées :
ID_element : L'identifiant de l'élément associé à l'alerte (par exemple, l'ID de la serre ou du capteur).
description : Une chaîne de caractères décrivant l'alerte (par exemple, "Température trop élevée").
priorite : Un entier représentant la priorité de l'alerte (plus le nombre est élevé, plus la priorité est haute).

Sortie :
Retourne un pointeur vers un nouvel objet alerte alloué dynamiquement, avec les informations initialisées.

Fonction : ajouter_alertes

Signature : `void ajouter_alerte(alertes **A,alerte *na)`

Rôle :
Cette fonction ajoute une alerte à la liste chaînée des alertes (alertes). L'alerte est insérée dans la liste en fonction de sa priorité (les alertes les plus prioritaires apparaissent en premier).

Entrées :
A : Un pointeur vers la tête de la liste chaînée des alertes.
na : Pointeur vers l'alerte à ajouter.

Sortie :
Aucune valeur de retour. La liste chaînée des alertes est modifiée en place.

Fonction : afficher_alertes

Signature : `void afficher_alertes(alertes *A)`

Rôle :
Cette fonction permet d'afficher toutes les alertes dans la liste chaînée. Elle parcourt la liste des alertes et imprime les détails (description, ID, priorité).

Entrées :
A : Pointeur vers la tête de la liste chaînée des alertes.

Sortie :
Aucune valeur de retour. Les alertes sont affichées à l'écran.

Fonction de gestion des notifications

Fonction : creat_notif

Signature : `notification *creat_notif(char *notif, int id_zone)`

Rôle :
Cette fonction crée une nouvelle notification en allouant dynamiquement de la mémoire et en initialisant ses champs.

Entrées :
notif : Une chaîne de caractères représentant l'action de la notification (par exemple, "Activation du chauffage").
id_zone : L'ID de la zone associée à la notification (par exemple, ID de la serre ou du champ).

Sortie :
Retourne un pointeur vers un nouvel objet notification alloué dynamiquement, avec les informations initialisées.

Fonction : Enfiler

Signature : `void Enfiler(file *F,notification *notif)`

Rôle :
Cette fonction ajoute une notification à la fin de la file de notifications. Si la file est vide, elle initialisera les pointeurs first et last. Sinon, elle insère la notification à la fin de la file.

Entrées :
F : Pointeur vers la file de notifications.
notif : Pointeur vers la notification à ajouter.

Sortie :
Aucune valeur de retour. La file de notifications est mise à jour.

Fonction : afficher_File

Signature : `void afficher_File(file *F)`

Rôle :
Cette fonction permet d'afficher le contenu de la file des notifications, en parcourant la liste et en imprimant chaque notification (action et ID de zone).

Entrées :
F : Pointeur vers la file de notifications.

Sortie :
Aucune valeur de retour. La liste des notifications est affichée à l'écran, ainsi que la taille de la file.

Fonctions pour la gestion des fichier d'historique

Fonction : fichier__serres

Signature : `void fichier__serres(serres *S, int id_serre)`

Rôle : Cette fonction enregistre l'historique des informations d'une serre, telles que la température, l'humidité, la pluie, l'irrigation, le chauffage et l'état, dans un fichier texte. Cela permet de suivre les données relatives à chaque serre au fil du temps.

Entrées :

S (serres*) : Une liste chaînée représentant l'ensemble des serres.

id_serre (int) : L'identifiant de la serre pour laquelle l'historique doit être enregistré.

Sorties :

Aucun retour, mais la fonction écrit les données de la serre dans le fichier "historique_serres.txt".

Autres Fonctions :

Les fonctions suivantes suivent la même logique que celle de fichier_serres, mais pour d'autres composants du système :

fichier__champs : `void fichier__champs(champs *CH, int id_champ)`

Enregistre l'historique des données d'un champ (température, humidité, pH, irrigation) dans le fichier "historique_champs.txt".

fichier__panneaux : `void fichier__panneaux(panneaux_sol *P, int id_panneau)`

Enregistre l'historique des données d'un panneau solaire (luminosité, ajustement, état) dans le fichier "historique_panneaux.txt".

fichier__reserves : `void fichier__reserves(reserves_eau *R, int id_reserve)`

Enregistre l'historique des données d'une réserve d'eau (niveau, fuite, pression, pompe) dans le fichier "historique_reserves.txt".

Fonction : get_current_timestamp

Signature : `void get_current_timestamp(char *times)`

Rôle : Cette fonction permet d'obtenir l'heure et la date actuelles du système sous un format spécifique. Elle est utilisée pour enregistrer l'heure dans les fichiers d'historique afin de suivre les événements dans le temps.

Entrées :

times (char*) : Un tableau de caractères où la date et l'heure formatées seront stockées.

Sorties :

Aucun retour, mais la variable times est modifiée pour contenir l'heure et la date actuelles au format "JJ/MM/AAAA-HH:MM:SS".

Fonctions d’affichage

Fonction : afficher_serres

Signature : `void afficher_serres(serres *S)`

Rôle :
permet d'afficher les informations détaillées sur chaque serre contenue dans une liste chaînée de structures serres. Elle parcourt cette liste et affiche les caractéristiques de chaque serre.(l'id de la serre, les valeurs des capteurs, l'état du système d'irrigation et de chauffage)

Entrées :
S : un pointeur vers une structure serres. Cette structure représente la liste chaînée des serre.

Sortie :
Aucune valeur de retour. Les serres sont affichées à l'écran.

Fonction : afficher_champs

Signature : `void afficher_champs(champs *CH)`

Rôle :
permet d'afficher les informations détaillées sur chaque champ contenue dans une liste chaînée de structures champs.

Entrées :
CH : un pointeur vers une structure champs. Cette structure représente la liste chaînée des champ.

Sortie :
Aucune valeur de retour. Les champs sont affichées à l'écran.

Fonction : afficher_panneaux

Signature : `void afficher_panneaux(panneaux_sol *P)`

Rôle :
permet d'afficher les informations détaillées sur chaque panneau solaires contenue dans une liste chaînée de structures panneaux.

Fonction : afficher_reserves

Signature : `void afficher_reserves(reserves_eau *R)`

Rôle :
permet d'afficher les informations détaillées sur chaque reserve d’eau contenue dans une liste chaînée de structures reserves_eau.

Fonctions d'ajout

Fonction : ajouter_serres

Signature :

```
void ajouter_serres(serres **S,serre *ns)
```

Rôle :
permet d'ajouter une nouvelle serre à la fin d'une liste chaînée de structures serres.
Entrées :
S : un pointeur vers la tête de la liste chaînée de serres.
Sortie :
Aucune valeur de retour . Elle modifie la liste chaînée pointée par S en ajoutant la serre ns à la fin de la liste.

---> cette fonction necessite les deux fonction suivantes :

Fonction : creat_capteur

Signature :

```
capteur *creat_capteur(int id_capteur,char *type)
```

Rôle :
permet de créer et initialiser un nouveau capteur. Elle alloue dynamiquement de la mémoire pour une structure capteur, attribue un identifiant (id) à ce capteur, définit son type (température,humidité,luminosité...), et initialise la valeur du capteur à 0.
Entrées :
id_capteur : un entier qui représente l'identifiant unique du capteur à créer.
type : une chaîne de caractères qui représente le type du capteur .
Sortie :
La fonction retourne un pointeur vers la nouvelle structure capteur .

Fonction : creat_serre

Signature :

```
serre *creat_serre(int id_serre,int id_temp,int id_hum,int id_pluit)
```

Rôle :
permet de créer et initialiser une nouvelle structure serre. Elle alloue dynamiquement de la mémoire pour cette structure, lui attribue un identifiant, initialise ses systèmes , crée et initialise les capteurs (température, humidité, pluie) associés à la serre, et définit son état initial (par défaut "ouvert").
Entrées :
id_serre : un entier représentant l'identifiant unique de la serre à créer.
id_temp : un entier représentant l'identifiant du capteur de température.
id_hum : un entier représentant l'identifiant du capteur d'humidité.
id_pluit : un entier représentant l'identifiant du capteur de pluie.

Sortie :
La fonction retourne un pointeur vers la nouvelle structure serre .

Autre fonctions :

Les fonctions suivantes suivent la même logique que celle de ajouter_serres, mais pour d'autres composants du système :

Fonction : ajouter_champs

Signature :
```
void ajouter_champs(champs **CH,champ_ouvert *nch)
```

Rôle : permet d'ajouter le nouveau champ à la fin d'une liste chaînée des champs.

Fonction : ajouter_panneaux

Signature :
```
void ajouter_panneaux(panneaux_sol **P,panneau_sol *np)
```

Rôle :permet d'ajouter le nouveau panneau solaire à la fin d'une liste chaînée des panneaux.

Fonction : ajouter_reserves

Signature :
```
void ajouter_reserves(reserves_eau **R,reserve_eau *nr)
```

Rôle : permet d'ajouter le nouveau reserve à la fin d'une liste chaînée des reserves.

Fonctions de suppression

Fonction : supprimer_serre

Signature :
```
void supprimer_serre(serres *s)
```

Rôle : Cette fonction supprime une serre spécifique en libérant la mémoire allouée pour la serre, ses capteurs et autres structures associées.

Fonction : supprimer_serres

Signature :
```
void supprimer_serres(serres **S,int id_serre)
```

Rôle :
Supprime une serre spécifique dans une liste chaînée de serres en utilisant l'ID de la serre, et libère la mémoire associée en appelant la fonction supprimer_serre mentionnée ci-dessus.
Entrées :
S : Pointeur vers un pointeur de la tête de liste de type serres.
id_serre : L'ID de la serre à supprimer.
Sorties :Aucun retour. La serre avec l'ID spécifié est supprimée et la mémoire est libérée.

Les autres fonctions de suppression suit la même logique

Fonction : supprimer_champ

Signature : `void supprimer_champ(champs *ch)`

Rôle : Supprime un champ spécifique et libère la mémoire associée au champ et ses capteurs.

Fonction : supprimer_champs

Signature : `void supprimer_champs(champs **CH,int id_champ)`

Rôle : Supprime un champ spécifique dans une liste chaînée de champs et libère la mémoire associée.

Fonction : supprimer_panneau

Signature : `void supprimer_panneau(panneaux_sol *p)`

Rôle : Supprime un panneau solaire spécifique et libère la mémoire associée au panneau et ses capteurs.

Fonction : supprimer_panneaux

Signature : `void supprimer_panneaux(panneaux_sol **P,int id_panneau)`

Rôle : Supprime un panneau solaire spécifique dans une liste chaînée de panneaux solaires et libère la mémoire associée.

Fonction : supprimer_reserve

Signature : `void supprimer_reserve(reserves_eau *r)`

Rôle : Supprime une réserve d'eau spécifique et libère la mémoire associée à la réserve et ses capteurs.

Fonction : supprimer_reserves

Signature : `void supprimer_reserves(reserves_eau **R,int id_reserve)`

Rôle : Supprime une réserve d'eau spécifique dans une liste chaînée de réserves d'eau et libère la mémoire associée.

Toutes les fonctions de suppression suivent une logique similaire pour libérer la mémoire allouée et ajuster les pointeurs dans les listes chaînées des différents composants.

Fonctions pour changer les valeurs des capteurs

Fonction : changer_cap_value_serre

Signature :

```
void changer_cap_value_serre(serres *S, int id_serre, int val_temperature, int val_humidite, int val_pluit)
```

Rôle : Cette fonction permet de changer les valeurs des capteurs de température, d'humidité et de pluie d'une serre donnée, en fonction de son identifiant. Si la valeur est différente de -1, elle sera mise à jour.
Entrées :
S (serres*) : Liste chaînée représentant toutes les serres.
id_serre (int) : L'identifiant de la serre pour laquelle les valeurs des capteurs doivent être modifiées.
val_temperature, val_humidite, val_pluit (int) : Les nouvelles valeurs des capteurs. Si la valeur est -1, le capteur ne sera pas modifié.
Sorties :
Aucun retour, mais la fonction modifie les valeurs des capteurs de la serre dans la liste.

Autres Fonctions:

Les fonctions suivantes suivent la même logique que celle de changer_cap_value_serre, mais pour d'autres composants du système :

changer_cap_value_champ :

```
void changer_cap_value_champ(champs *C, int id_champ, int val_temperature, int val_humidite, int val_ph)
```

Permet de changer les valeurs des capteurs de température, d'humidité et de pH dans un champ donné.

changer_cap_value_panneau :

```
void changer_cap_value_panneau(panneaux_sol *P,int id_panneau,int nvl_val)
```

Modifie la valeur du capteur de luminosité d'un panneau solaire, en fonction de son identifiant.

changer_cap_value_reserve :

```
void changer_cap_value_reserve(reserves_eau *R,int id_reserve,int val_niveau,int val_fuit,int val_pression)
```

Change les valeurs des capteurs de niveau d'eau, de fuite et de pression dans une réserve d'eau.

Fonctions d’ajustement

1. Fonction : ajustement_panneau_sol

Signature : `void ajustement_panneau_sol(panneaux_sol *P,file *file_notif)`

Rôle : Ajuste le comportement des panneaux solaires en fonction de la luminosité mesurée. En fonction de la luminosité, il active ou désactive l'ajustement du panneau solaire et enregistre des notifications.

Entrées:

P (panneaux_sol*) : Liste des panneaux solaires.

file_notif (file*) : File de notifications pour enregistrer l'état des panneaux solaires.

Sorties:

Aucune valeur retournée, mais des notifications sont ajoutées à la file.

2. Fonction : ajustement_serres

Signature : `void ajustement_serres(serres *S,alertes **A,file *file_notif)`

Rôle : Gère les systèmes d'irrigation et de chauffage des serres selon la température, l'humidité et les précipitations. Ajoute des alertes si nécessaire et enregistre des notifications.

Entrées:

S (serres*) : Liste des serres.

A (alertes**) : Liste des alertes à ajouter en cas de conditions extrêmes.

file_notif (file*) : File de notifications pour enregistrer les actions entreprises.

Sorties:

Aucune valeur retournée, mais des alertes et notifications sont ajoutées.

3. Fonction : ajustement_reserves

Signature : `void ajustement_reserves(reserves_eau *R,alertes **A,file *file_notif)`

Rôle : Contrôle les réservoirs d'eau en fonction du niveau d'eau, de la fuite et de la pression. Active ou désactive la pompe de remplissage et la vanne de décharge. Déclenche des alertes si des fuites ou des pressions élevées sont détectées.

Entrées:

R (reserves_eau*) : Liste des réserves d'eau.

A (alertes**) : Liste des alertes à ajouter.

file_notif (file*) : File de notifications.

Sorties:

Aucune valeur retournée, mais des alertes et notifications sont générées.

4. Fonction : ajustement_champs

Signature : `void ajustement_champs(champs *C,alertes **A,file *file_notif)`

Rôle : Gère le système d'irrigation des champs en fonction de la température et de l'humidité. Déclenche des alertes si l'humidité ou le pH dépasse les seuils optimaux.

Entrées:

C (champs*) : Liste des champs.

A (alertes**) : Liste des alertes.

file_notif (file*) : File de notifications.

Sorties:

Aucune valeur retournée, mais des alertes et notifications sont générées.

Fonctions de gestion

1. Fonction gestion_serres

Prototype : `void gestion_serres(serres **S,alertes **A,file *file_notif)`

Rôle : Cette fonction permet de gérer les actions possibles sur les serres .
Options disponibles :

- Afficher les serres.
- Ajouter une serre avec ses capteurs (température, humidité, pluie).
- Supprimer une serre existante.
- Modifier les valeurs des capteurs d'une serre et ajuster automatiquement les systèmes d'irrigation et de chauffage.

Entrées :
S (serres**) : Pointeur vers la liste des serres.
A (alertes**) : Liste des alertes à gérer, ajoutées si nécessaire.
file_notif (file*) : File de notifications pour enregistrer les actions effectuées.
Sorties :
Aucune valeur retournée, mais des notifications peuvent être ajoutées à file_notif et les valeurs des capteurs sont enregistrées dans un fichier spécifique au serres.

Les fonctions de gestion des champs, des panneaux solaires, des réserves d'eau et de la ferme suivent une logique similaire à celle de la fonction gestion_serres.

gestion_champs : Permet de gérer les champs, incluant l'ajout, la suppression ,l'affichage des champs et la modification de valeurs des capteurs (température, humidité, pH) .

Prototype : `void gestion_champs(champs **CH,alertes **A,file *file_notif)`

gestion_panneaux : Permet de gérer les panneaux solaires,incluant l'ajout, la suppression ,l'affichage des panneaux solaires et la modification de valeur de capteur de luminosité

Prototype : `void gestion_panneaux(panneaux_sol **P,file *file_notif)`

gestion_reserves_eau : Permet de gérer les réserves d'eau, incluant l'ajout,la suppression, l'affichage des réserves d'eau et la modification de valeurs des capteurs (niveau, fuite, pression).

Prototype : `void gestion_reserves_eau(reserves_eau **R,alertes **A,file *file_notif)`

gestion_ferme : Permet Accès à la gestion des serres, champs, réserves d'eau et panneaux solaires et l'affichage des notifications générées lors des ajustements automatiques.

Prototyp : `void gestion_ferme(ferme *F,alertes **A,file *file_notif)`

fonction main()

Rôle : permet de gèrer l'initialisation de la ferme et de la file des notifications et la liste des alertes . Elle permet la gestion de la ferme et de libérer la mémoire allouée après l'exécution.

Description des étapes :

1) Initialisation de la structure ferme et de la file des notifications et des alertes.

```
main() {

    ferme *F=(ferme*)malloc(sizeof(ferme));
    F->serres=0;
    F->champs=0;
    F->reserves_eau=0;
    F->panneaux_sol=0;
    alertes *A=NULL;
    file *file_notif=(file*)malloc(sizeof(file));
    file_notif->first=NULL;
    file_notif->last=NULL;
    file_notif->size=0;
```

2) Appel à la fonction gestion_ferme() pour gérer la ferme.
3) Boucle permettant à l'utilisateur de gérer la ferme tant qu'il entre le choix 1.

```
    printf("-----gestion de la ferme-----\n");
    int choix;
    printf("clique 1 pour la gestion de la ferme :");
    scanf("%d",&choix);
    while(choix==1){
    gestion_ferme(F,&A,file_notif);
    printf("clique 1 pour la gestion de la ferme :");
    scanf("%d",&choix);
    }
```

4) Libération de la mémoire allouée pour chaque élément de la ferme (serres, champs, panneaux solaires, réserves d'eau).

```
    //liberer l'espace mémoire :
    serres *i=F->serres;
    while(i!=NULL){
        serres *tmp1=i;
        i=i->next;
        supprimer_serre(tmp1);
    }
    champs *j=F->champs;
    while(j!=NULL){
        champs *tmp2=j;
        j=j->next;
        supprimer_champ(tmp2);
    }
    panneaux_sol *k=F->panneaux_sol;
    while(k!=NULL){
        panneaux_sol *tmp3=k;
        k=k->next;
        supprimer_panneau(tmp3);
    }
    reserves_eau *l=F->reserves_eau;
    while(l!=NULL){
        reserves_eau *tmp4=l;
        l=l->next;
        supprimer_reserve(tmp4);
    }
```

DIFFICULTÉS RENCONTRÉES

Difficulté	Problème	Solution	Impact
Gestion de la mémoire dynamique	Difficulté d'allocation/libération de mémoire pour les structures complexes.	Utilisation de malloc et free pour chaque allocation. Création de fonctions dédiées pour libérer la mémoire. Vérification des pointeurs.	Ralentissement du développement, tests approfondis nécessaires pour éviter les fuites mémoire.
Gestion des alertes et notifications	Logique complexe pour trier et gérer les alertes et notifications.	Création d'une file pour notifications et d'une liste chaînée pour alertes. Tri par priorité avec ajouter_alerte, ajout à la file avec Enfiler.	Complexité supplémentaire mais centralisation des informations importantes.
Gestion des structures complexes	Complexité dans la gestion des structures imbriquées (ex :serres avec capteurs).	Modularisation du code : décomposition en fonctions distinctes pour chaque tâche (par exemple, creat_serre, afficher_serres). Ajout de commentaires pour clarifier le rôle de chaque fonction. Tests unitaires pour valider chaque fonction indépendamment.	Le code devient plus lisible, maintenable, et testable
Manque de compétences en bases de données	Difficulté à implémenter une solution de gestion des données avec des bases de données en raison du manque de compétences	Le choix a été fait de stocker les données dans des fichiers texte pour simplifier le projet. Une réflexion sur l'utilisation d'une base de données pourrait être envisagée à l'avenir.	Cela a entravé la possibilité d'avoir une gestion des données plus robuste, et le stockage dans des fichiers texte a limité la flexibilité du système.
Manque d'interface graphique	Absence d'interface graphique pour une meilleure interaction avec l'utilisateur.	Le manque de compétences actuelles en java a conduit à une interface uniquement textuelle.	L'absence d'interface graphique a limité l'expérience utilisateur, rendant l'interaction moins intuitive et agréable.

PERSPECTIVES

Améliorations possibles du projet :

Amélioration de l'interface utilisateur :

Nous pourrions améliorer l'interface graphique pour rendre la navigation plus fluide et agréable. Par exemple, une carte interactive de la ferme avec des éléments visuels animés qui montrent en temps réel l'impact des changements climatiques. Cela rendrait l'expérience plus immersive pour l'utilisateur.En utilisant Java avec une bibliothèque comme JavaFX ou Swing, nous pourrions développer une interface plus fluide et intuitive, avec des éléments interactifs (curseurs pour ajuster les conditions climatiques, boutons pour activer/désactiver certains systèmes, etc.).

Simulation plus réaliste des capteurs :

Actuellement, les capteurs sont simulés. Une amélioration possible serait de rendre ces capteurs plus réalistes en générant des données plus proches de celles que l'on retrouverait dans la réalité. Cela permettrait de rendre les réactions des systèmes de la ferme plus crédibles et mieux ajustées aux conditions climatiques simulées.

Prédictions avec Machine Learning :

Une fonctionnalité de prédiction basée sur le Machine Learning pourrait être intéressante pour anticiper les conditions climatiques à venir et ajuster les systèmes automatiquement. Par exemple, prévoir une période de sécheresse et ajuster les niveaux d'irrigation en conséquence.

Intégration d'une base de données :

À l'avenir, il serait judicieux d'ajouter une base de données (comme MySQL ou MongoDB) pour centraliser toutes les informations relatives aux capteurs, alertes, et historiques des données. Cela nous permettrait de mieux gérer les grandes quantités de données et d'optimiser la gestion de la ferme.

CONCLUSION

Dans ce projet, nous avons étudié et développé une solution innovante pour la gestion automatisée des fermes intelligentes en utilisant les structures de données avancées du langage C. Grâce à une architecture modulaire et flexible, nous avons pu intégrer divers composants tels que les serres, les champs ouverts, les panneaux solaires et les réservoirs d'eau, tout en prenant en compte les capteurs simulés et les systèmes d'alerte et de notification.

Ce projet nous a permis de mieux comprendre les défis liés à la gestion des ressources dans le secteur agricole et de proposer des solutions concrètes pour optimiser l'utilisation des ressources comme l'eau et l'énergie. Bien que le système soit fonctionnel dans un environnement simulé, des améliorations restent possibles, notamment l'intégration d'une interface graphique en Java, l'utilisation de bases de données pour une gestion plus robuste des données, ainsi que l'ajout de modèles prédictifs basés sur des algorithmes de machine learning.

En conclusion, ce projet constitue une contribution significative à la modernisation de la gestion agricole tout en renforçant nos compétences dans la gestion des structures de données et le développement de systèmes automatisés. Nous espérons que ce travail pourra servir de base pour des développements futurs visant à rendre les fermes intelligentes encore plus autonomes, durables et efficaces.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à notre encadrant, M. Abdelwahab Naji, pour sa patience, son soutien et ses conseils précieux tout au long de ce projet. Ses orientations nous ont permis de clarifier nos idées et de surmonter les défis rencontrés.